

Additively Competitive Secretaries

Mohammad Mahdian

Google Research
New York, NY, USA
mahdian@google.com

Jieming Mao

Google Research
New York, NY, USA
maojm@google.com

Enze Sun

School of Computing and Data
Science
The University of Hong Kong
Hong Kong, China
sunenze@connect.hku.hk

Kangning Wang

Department of Computer Science
Rutgers University
Piscataway, NJ, USA
kn.w@rutgers.edu

Yifan Wang

School of Computer Science
Georgia Institute of Technology
Atlanta, Georgia, USA
ywang3782@gatech.edu

Abstract

In the secretary problem, a set of secretary candidates arrive in a uniformly random order and reveal their values one by one. A company, who can only hire one candidate and hopes to maximize the expected value of its hire, needs to make irrevocable online decisions about whether to hire the current candidate. The classical framework of evaluating a policy is to compute its worst-case competitive ratio against the optimal solution in hindsight, and there the best policy – the “ $1/e$ law” – has a competitive ratio of $1/e$.

We propose an alternative evaluation framework through the lens of *regret* – the worst-case additive difference between the optimal hindsight solution and the expected performance of the policy, assuming that each value is normalized between 0 and 1. The $1/e$ law for the classical framework has a regret of $1 - 1/e \approx 0.632$; by contrast, we show that the class of “pricing curves” algorithms can guarantee a regret of at most $1/4 = 0.25$ (which is tight within the class), and the class of “best-only pricing curves” algorithms can guarantee a regret of at most 0.190 (with a lower bound of 0.171). In addition, we show that in general, no policy can give a regret guarantee better than 0.152. Finally, we discuss other objectives in our regret-minimization framework.

CCS Concepts

• **Theory of computation** → **Online algorithms**; *Algorithmic game theory*.

Keywords

Secretary Problem, Regret Minimization, Online Algorithm

Enze Sun is supported by RGC Hong Kong (No. 17211723).



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792478>

ACM Reference Format:

Mohammad Mahdian, Jieming Mao, Enze Sun, Kangning Wang, and Yifan Wang. 2026. Additively Competitive Secretaries. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792478>

1 Introduction

In the domain of online decision making, people constantly face irrevocable decisions amid an uncertain future, where information about that uncertain future unfolds gradually. Central to this area is the theory of optimal stopping – selecting one time to stop in order to maximize the associated reward. The *prophet inequality* and the *secretary problem* are two of the most prominent models of optimal stopping, each spawning numerous variants and fostering dedicated research in recent years.

In both the prophet inequality and the secretary problem, n values are observed sequentially by a decision maker, who needs to select one value by making irrevocable decisions on whether to take the current value or not. In the prophet inequality, the values are drawn from independent prior distributions, which are known upfront to the decision maker. In the secretary problem – a metaphor for a company trying to hire the best secretary – the values are instead unknown and adversarial, but their arrival order is uniformly random. Traditionally, in both frameworks, the quality of a stopping rule (i.e., an algorithm or a policy) is measured through the *competitive ratio* – the worst-case ratio between the (expected) performance of the algorithm and the (expected) optimum in hindsight. The competitive ratio of the optimal algorithm in the prophet inequality is $1/2$, and that in the secretary problem is $1/e$.

In this work, we depart from the traditional competitive approach, and examine a different evaluation framework through the notion of *regret*. Concretely, we assume that every value is within a certain known range, normalized¹ to $[0, 1]$, and naturally define the

¹This normalization is without loss of generality: if the value is known to be within a different range $[a, b]$, then all our results hold with regret multiplied by $b - a$ via linearly rescaling the instances and algorithms.

regret as the worst-case *additive difference* between the hindsight optimum and the performance of the algorithm. In fact, this framework of evaluating optimal stopping problems via regret has been proposed decades ago by Hill and Kertz, where they showed that the optimal regret is $1/4 = 0.25$ for the prophet inequality [32], and further examined various related settings [31, 33, 34]. However, to the best of our knowledge, there has not been any work in the literature that addresses the secretary problem in this additive-evaluation framework. Consequently, we explore the regret minimization objective for the secretary (random order) setting.

Why is this additive-evaluation framework valuable, especially in light of the extensive research on the competitive ratio of the secretary problem? We start by noting that our choice of studying this framework is influenced by its simplicity and elegance. We believe that our framework has the potential to inspire the development of novel algorithms, which, in turn, may offer practical utility or provide insights in real-world scenarios. Indeed, in many potential application domains, the (rough) knowledge about the range of possible values is available. The standard algorithm for the secretary problem – the “ $1/e$ law” – has a regret of $1 - 1/e \approx 0.632$ in our framework,² while our novel algorithms are significantly better, with regret guarantees as low as 0.190. Additionally, in many situations, especially when the values correspond to money (which is inherently additive), minimizing the additive regret makes more sense than maximizing the competitive ratio. Regret as a benchmark is also robust when making multiple independent decisions, where the sum of regrets is still a meaningful measurement. We believe that our work serves as a starting point for a future line of research, on re-examining classical online decision models from this different perspective of regret minimization.

1.1 Our Contribution

Our first contribution is to introduce the natural regret minimization framework for the secretary problem, formally in section 2. Then we begin our analysis by examining the regret of *pricing curves*. A pricing curve is an algorithm that specifies a threshold which only depends on the arrival time of the current value, and the algorithm accepts a value if it is above the threshold. In section 3, we show that setting $f(t) = 1 - t$ to be the threshold function of a pricing curve gives a $1/4 = 0.25$ regret bound. This implies that the *random-order arrival* in the secretary problem makes the regret minimization problem easier than adversarial-order arrival [34] (discussed in section 1.2). Our proof hinges on the observation that the “hardest” instance for such an algorithm only contains two different values, which allows us to compute the regret directly. We also show a matching lower bound of 0.25 for the class of pricing curves, meaning that we have to look beyond it to further improve the regret.

Next in section 4, as our main result, we identify another class of simple algorithms that have good performance and simultaneously

are amenable to analysis. We call them the *best-only pricing curves*. Such an algorithm again specifies a threshold that only depends on the arrival time of the current value, and then it accepts a value if both (1) it is above the threshold, and (2) it is the highest value seen so far. We give an upper bound of 0.190 and a lower bound of 0.171 for the optimal best-only pricing curve. The key for these bounds is a series of observations that lead to relaxed expressions of regret with only a small number of variables.

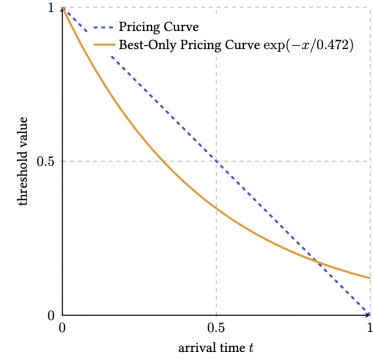


Figure 1: Optimal pricing curves for both algorithms.

Let us look further into a best-only pricing curve, in order to intuitively understand why it can provide improvement over pricing curves. In Figure 1, we illustrate the threshold function of the optimal pricing curve (blue dashed) and that of the best-only pricing curve with regret at most 0.190 (yellow solid). Firstly, note that the latter has a lower threshold function, which enables it to pick a sufficiently large value in cases where all values are small. Secondly, the best-only pricing curve uses the previously-seen maximum value as a second threshold, allowing the algorithm to “postpone” the time of choosing a frequently appearing value and hence giving the algorithm more chances to accept a higher value.

As a concrete example, consider an instance with a single value of 1 and near-infinitely many values of 0.5 with small noises to break ties. The pricing curve algorithm will accept the value of 1 if it arrives before $t = 0.5$, and otherwise will take a value of 0.5. This leads to its relatively large regret of 0.25. For the best-only pricing curve, although the threshold function drops below 0.5 at time $t \approx 0.3$, the early-arriving values of 0.5 with more positive noises act as a second threshold and prevent the algorithm from choosing a value of 0.5 with a less positive noise. As a result, with more than 66% chance, the algorithm skips all values of 0.5 and chooses the value of 1, improving the regret to less than 0.17.

To complement our positive results, in section 5, we provide a general lower bound of 0.152 that holds for any algorithm. The main idea of proving the lower bound is to show that the optimal algorithm has a relatively simple structure, albeit possibly with a large number of parameters. Then, we construct a hard instance with a manageable small size, compute the optimal (exponential time) algorithm for the hard instance, and show that the regret is at least 0.152.

Finally, in section 6, we discuss also the related objective of *revenue maximization* in the regret minimization framework. In

²The regret of the “ $1/e$ law” is at most $1 - 1/e$ because its expected performance is at least $1/e$ times the optimum, and the optimum is at most 1. Its regret is close to $1 - 1/e$ when there is a single value 1 and many values close to 0. Furthermore, this hard instance applies to all “ α -law” algorithms with $\alpha \in [0, 1]$, indicating that $1 - 1/e$ is the best possible regret for the class of algorithms that first wait some fraction of time and, then, accept the first value above the maximum in the waiting phase.

the revenue maximization problem, a seller is selling an item to n sequential buyers, where buyer i has valuation $x_i \in [0, 1]$ for the item. The seller needs to post a price of p_i before buyer i arrives. The first buyer with $x_i \geq p_i$ takes the item, and contributes p_i to the revenue. The goal of the seller is again to minimize the regret – the difference between the highest possible revenue, which is the maximum of all x_i 's, and the revenue achieved by the algorithm. For this objective, we show that the optimal regret is tight at $1/e$ for many variants of the model: the buyers' valuations can be either adversarial, stochastic, or i.i.d. stochastic, and the arrival order can be adversarial or uniformly random.

1.2 Further Related Works

Regret Minimization for Optimal Stopping Problems. The objective of minimizing regret in optimal stopping has a long history, beginning with the series of works by Hill and Kertz [31–34], who analyzed the standard, i.i.d., correlated, and order-selection versions of the prophet inequality. In the correlated case, [34] proved that when values are normalized to $[0, 1]$, the optimal regret of any stopping rule is $1/e$. For independent distributions, [32] established a tight regret bound of $1/4$, while in the i.i.d. setting, the optimal bound improves to approximately 0.111 [33]. These lower bounds naturally extend to our framework as special cases, although our analysis yields a stronger lower bound of 0.152, demonstrating a strictly harder setting.

Another line of research focuses on the *multisecretary problem*, initiated by Arlotto and Gurvich [2]. In this model, a sequence of T i.i.d. values drawn from a fixed distribution (independent of T) arrives online, and the decision maker may select up to k of them, often with k proportional to T . The objective is to minimize the *regret*, defined as the difference between the expected offline optimum and the algorithm's expected reward, in the limit as $T \rightarrow \infty$. There are a number of follow-up papers exploring this direction, including [7, 9, 11, 48], which study extensions to multi-type settings, continuous distributions, and other online allocation frameworks in operations research.

The Secretary Problem and Other Random-Order Models. The classical secretary problem dates back to early works such as [19, 26, 27, 43]. A wide range of variants have been explored, including minimizing relative rank [15], robust mixtures of random and adversarial orders [8], and models with advice or samples [16, 36]. The model has also been extended to combinatorial domains. Examples include the matroid secretary problem [4, 5, 12, 25, 42], and its constant-competitive versions for special matroid classes [18, 35]. Other generalizations consider bipartite and general matchings [23, 37, 39], downward-closed systems [45], and submodular objectives [6]. More recently, the *prophet secretary* model—combining prophet inequalities with random arrival—has attracted significant attention [21]. Improved competitive ratios have been obtained in successive works [3, 17, 29], while hardness bounds are given in [10, 17, 28]. Extensions to matroids and combinatorial auctions have also been studied [20].

Other Optimal Stopping Problems. The *prophet inequality*, originating from classical works [32, 40, 41, 47]—is a cornerstone in

optimal stopping and mechanism design, with numerous extensions to matroids, matchings, and other combinatorial domains [13, 22, 24, 38, 46]. A closely related line of research studies *revenue maximization* through sequential posted pricing (SPP), where each arriving buyer faces a take-it-or-leave-it price. Variants of SPP achieve constant-factor approximations to Myerson's optimal mechanism for single-item sales [1, 13, 30, 49] and extend to multi-item settings with unit-demand buyers [13, 14, 38].

2 Preliminaries

Throughout our work, we focus on the following definition of the secretary problem. An adversary specifies n values $x_1, x_2, \dots, x_n$ with $1 \geq x_1 \geq x_2 \geq \dots \geq x_n \geq 0$. Each value x_i arrives at time t_i drawn from $U[0, 1]$, the uniform distribution supported on the interval $[0, 1]$. Note that t_i 's are almost surely distinct. An online algorithm observes each value and its arrival time³ in the increasing order of their arrival time, and needs to immediately decide whether to pick the observed value or to discard it. An algorithm can only pick one value, and for consistency, we assume that an algorithm always picks a value of 0 at the end if it has not picked any value yet. (Alternatively, we can always modify an algorithm so that it never skips the last value.)

Let $\text{Alg}(\vec{x})$ be the expectation of the picked value of the algorithm Alg when the values are $\vec{x}$. Our objective is to minimize the worst-case *regret*, which is defined as

$$\text{regret} := \sup_{\vec{x}} \text{regret}(\vec{x}), \quad \text{where} \quad \text{regret}(\vec{x}) := x_1 - \mathbb{E}[\text{Alg}(\vec{x})].$$

We note that values arriving at random time drawn from $U[0, 1]$ is equivalent to values arriving in a uniformly random order, since the time vector beyond the order does not provide any useful information. An algorithm that uses this information in its description can simply freshly draw the vector of arrival time from $(U[0, 1])^n$. We note that the step to simplify the analysis has been done in related work, such as that of [16, 17].

3 Warm-up: The Regret of Pricing Curves

In this section, we discuss *pricing curves* – a class of simple algorithms that, as we will show, can achieve a regret bound of 0.25. A pricing curve specifies a (measurable) threshold function $f(t): [0, 1] \rightarrow [0, 1]$ that defines a threshold value at each possible arrival time. It then picks the first-arrived value that is above the threshold at its arrival time.

Theorem 3.1. *Algorithm 1 with $f(t) = 1 - t$ achieves a regret of 0.25.*

PROOF. We consider a more difficult variant of our setting, and show that the regret of the pricing curve with $f(t) = 1 - t$ is at most 0.25 in that variant, hence implying our theorem statement.

In the variant, x_1 arrives at $t_1 \sim U[0, 1]$, but $t_2, t_3, \dots, t_n$ are determined upfront by an adversary. We now characterize the hardest instance for our algorithm in this variant.

³In fact, the algorithm does not need to observe the arrival time of the current value: it can compute the distribution of the arrival time and sample from it. This is a known technique to simplify the technical derivations; see, e.g., [17].

Algorithm 1: Pricing Curve

```

Let  $f(t) : [0, 1] \rightarrow [0, 1]$  be its threshold function
for  $\tau = 0 \rightarrow 1$  where  $x_i$  arrives at  $t_i = \tau$  do
    if  $x_i \geq f(t_i)$  then
        | Pick  $x_i$  and exit
    else
        | Skip  $x_i$ 
    end
end

```

First, notice that if $x_i < f(t_i)$ for some $i \geq 2$, we can simply remove this x_i without changing the behavior or the regret of our algorithm. We do this for all such x_i 's.

After that, let $j := \arg \min_{i \geq 2} t_i$. (If all x_i 's with $i \geq 2$ have been removed and hence the index j does not exist, we can add back $x_2 = 0$ with $t_2 = 1$ without changing the analysis.) Again, we can remove all values except x_1 and x_j without changing the behavior or the regret of our algorithm.

Now we analyze the regret after these simplifications. Observe that our algorithm picks x_1 if and only if $t_1 \in [1 - x_1, t_j]$, and picks x_j otherwise. Therefore, the regret of our algorithm is

$$\text{regret} = (x_1 - x_j) \cdot (1 - (t_j - (1 - x_1))) \leq (x_1 - x_j) \cdot (1 - x_1 + x_j) \leq 0.25,$$

where in the first inequality, we use the fact that $x_j \geq 1 - t_j$. $\square$

Theorem 3.1 shows that a pricing curve with a simple threshold function achieves a lower regret than that of $1 - 1/e$ of the “ $1/e$ law”, and the optimal regret of $1/e$ in the adversarial-order setting. As it turns out (in theorem 3.2), this regret bound of 0.25 is optimal among all pricing curves.

Theorem 3.2. *Algorithm 1 has a regret of at least 0.25 for any pricing curve $f(t)$.*

PROOF. Consider the following two instances:

- Instance (1): $n = 1$ and $x_1 = 0.5$.
- Instance family (2): $x_1 = 1$ and $x_2 = \dots = x_n = 0.5$.

We will show that Algorithm 1 with any fixed threshold function $f(t)$ has a regret of at least 0.25 for either Instance (1) or Instance family (2).

Let $\alpha := \mu(\{t \in [0, 1] : f(t) < 0.5\})$ denote the fraction (Lebesgue measure) of time where a value of 0.5 is above the threshold. If $\alpha \leq 0.5$, then the pricing curve incurs regret of at least $(1 - \alpha)x_1 \geq 0.25$ in Instance (1). Otherwise (if $\alpha > 0.5$), note that there is a probability of 0.5 that $x_1 = 1$ arrives at $t_1 \geq 0.5$ in Instance family (2). However, since $\mu(\{t \in [0, 0.5] : f(t) < 0.5\}) \geq \alpha - 0.5 > 0$, we know that as $n \rightarrow \infty$, the probability that the pricing curve has accepted some value by time $t = 0.5$ approaches 1. Therefore, the regret of the pricing curve in Instance family (2) is at least $0.5(x_1 - x_2) = 0.25$. $\square$

4 Main Result: The Regret of Best-Only Pricing Curves

By now, we have seen that the pricing curves can achieve a regret of 0.25 but not better. That leads to a natural question: Can we achieve a lower regret using other – hopefully still simple – algorithms?

In this section, we move our attention to the class of *best-only pricing curves*, and show that the regret of one such algorithm is at most 0.190. We complement our result by a lower bound of 0.171 for any algorithm in this class.

Similar to a pricing curve, a best-only pricing curve specifies a threshold function $f(t) : [0, 1] \rightarrow [0, 1]$. When the value x_i arrives at time t_i , we accept it if and only if both

- $x_i \geq f(t_i)$, and
- x_i is the largest value that has arrived so far.⁴

The following Algorithm 2 formally defines the algorithm we run. Note that it is possible that Algorithm 2 skips all values. In this case, we assume that the value we choose is 0 and the regret is x_1 .

Algorithm 2: Best-Only Pricing Curve

```

Let  $f(t) : [0, 1] \rightarrow [0, 1]$  be its threshold function
for  $\tau = 0 \rightarrow 1$  where  $x_i$  arrives at  $t_i = \tau$  do
    if  $x_i \geq f(t_i)$  and  $x_i$  is the largest value that has arrived so far then
        | Pick  $x_i$  and exit
    else
        | Skip  $x_i$ 
    end
end

```

4.1 Regret Relaxation

Fix an instance with values $x_1, \dots, x_n$, where $1 \geq x_1 \geq x_2 \geq \dots \geq x_n \geq 0$. We first provide some preliminary analysis for Algorithm 2, which will indicate that the hard instance for Algorithm 2 after relaxing the regret function has a relatively simple structure.

For simplicity of our analysis, in this section, we only consider Algorithm 2 with a pricing curve $f(t)$ with the following properties:

- $f(t)$ is continuous and monotonically strictly decreasing.
- $f(0) = 1$.

With the above assumptions, the following observation suggests that it is sufficient only to consider the instances with values $x_1 \geq x_2 \geq \dots \geq x_n \geq f(1)$.

Observation 4.1. *Fix the pricing curve $f(t)$ that satisfies continuity, monotonicity, and that $f(0) = 1$. Let $x_1 \geq x_2 \geq \dots \geq x_n$ be an input instance that maximizes the expected regret of Algorithm 2 with pricing curve $f(t)$. Then, for any positive integer k , there exists a setting of $\vec{x} = (x_1, \dots, x_n)$ that satisfies:*

- $n \geq k$.

⁴To avoid tie-breaking, we add an arbitrarily small noise to each value x_i , which makes all values almost surely unique, without affecting the performance guarantee of the algorithm.

- $x_1 \geq x_2 \geq \dots \geq x_n \geq f(1)$.

PROOF. It is sufficient to show that for any instance that does not satisfy the properties in Observation 4.1, there exists another instance with a regret no smaller than the current instance. Fix $\vec{x} = (x_1, \dots, x_n)$ to be an instance that does not satisfy the above properties. Consider going through the following three inspections sequentially.

- *Inspection 1:* $x_1 < f(1)$. Note that in this case, the regret of the instance is $x_1 < f(1)$. In this case, we update x_1 to $f(1)$, and the regret becomes $f(1)$.
- *Inspection 2:* $x_n < f(1)$. In this case, we remove x_n from the instance, and the regret of the instance remains unchanged. We ask the instance to go through Inspection 2 multiple times.
- *Inspection 3:* $n \geq 1$ but $n < k$. In this case, we add multiple $f(1)$ into the instance until $n \geq k$. Note that we require $f(t)$ to be strictly decreasing. Therefore, a value $f(1)$ can only be chosen with probability 0, and adding a constant number of $f(1)$ into the instance does not change the regret.

Note that after passing the above three inspections, the regret of the instance does not decrease, but the properties in Observation 4.1 are satisfied, which finishes the proof. $\square$

With Observation 4.1, in the following of the section we only consider input instance $x_1 \geq x_2 \geq \dots \geq x_n \geq f(1)$ with $n \geq 10$.

Let $g(x) : [f(1), 1] \rightarrow [0, 1]$ be the “inverse function” of $f(t)$, where, formally, $g(x) := \inf\{t \in [0, 1] : x \geq f(t)\}$. In other words, $g(x_i)$ is the earliest time that the algorithm can accept x_i . (Note that $t_i \neq g(x_i)$ almost surely, and hence the corner case where $t_i = g(x_i)$ can be ignored.) For simplicity, we use θ_i to denote $g(x_i)$, the earliest time that the algorithm can accept x_i . We have $\theta_n = 1$.

Next, we give a regret upper-bound for the instance $\vec{x}$. Perhaps surprisingly, we can simplify the regret for a fixed instance into the more manageable expression in lemma 4.2.

Lemma 4.2. *It holds that*

$$\text{regret}(\vec{x}) = x_1 \cdot \theta_1 + \left(\sum_{i=2}^{n-1} (x_1 - x_i) \cdot \frac{1}{i} \cdot (1 - \theta_i)^i \right) - \left(\sum_{i=2}^{n-1} (x_1 - x_i) \cdot \sum_{k=i+1}^{n-1} \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1 - \theta_k)^k \right).$$

PROOF. Let A_0 be the event that the algorithm does not accept any value. This event A_0 occurs if and only if x_1 arrives before θ_1 , otherwise we must accept some value before or at the time that x_1 arrives. Therefore, $\Pr[A_0] = \theta_1$.

Let A_i be the event that the algorithm accepts x_i . Now we fix i and calculate the probability that A_i occurs. Recall that $t_i \sim U[0, 1]$ is the arrival time of x_i . If $t_i < \theta_i$, then x_i will be rejected. Otherwise, we have $t_i \in [\theta_k, \theta_{k+1})$ for some $k \geq i$. When A_i occurs, there are two cases:

- (1) For all $s \in [k] \setminus \{i\}$, the value x_s arrives after t_i . This happens with probability $(1 - t_i)^{k-1}$.

- (2) Otherwise, let $j := \arg \min\{s \in [k] \setminus \{i\} : x_s \text{ arrives before } t_i\}$. In this case, A_i occurs if and only if $j > i$ and x_j arrives before θ_j . This probability is $(1 - t_i)^{j-2} \cdot \theta_j$.

Therefore, through algebraic manipulation, we have

$$\begin{aligned} \Pr[A_i] &= \sum_{k=i}^{n-1} \int_{\theta_k}^{\theta_{k+1}} \left((1 - \tau)^{k-1} + \sum_{j=i+1}^k \theta_j \cdot (1 - \tau)^{j-2} \right) d\tau \\ &= \left(\sum_{k=i}^{n-1} \int_{\theta_k}^{\theta_{k+1}} (1 - \tau)^{k-1} d\tau \right) + \left(\sum_{j=i+1}^{n-1} \theta_j \cdot \int_{\theta_j}^1 (1 - \tau)^{j-2} d\tau \right) \\ &= \left(\sum_{k=i}^{n-1} \frac{1}{k} \left((1 - \theta_k)^k - (1 - \theta_{k+1})^k \right) \right) \\ &\quad + \left(\sum_{j=i+1}^{n-1} \frac{\theta_j}{j-1} \cdot (1 - \theta_j)^{j-1} \right) \\ &= \sum_{k=i}^{n-1} \frac{1}{k} \left((1 - \theta_k)^k - (1 - \theta_{k+1})^k + \theta_{k+1} (1 - \theta_{k+1})^k \right) \\ &= \sum_{k=i}^{n-1} \frac{1}{k} \left((1 - \theta_k)^k - (1 - \theta_{k+1})^{k+1} \right) \\ &= \frac{1}{i} \cdot (1 - \theta_i)^i - \sum_{k=i+1}^{n-1} \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1 - \theta_k)^k. \end{aligned}$$

Now we calculate the regret of our algorithm by plugging in these formulas. Recall that $\theta_n = 1$.

$$\begin{aligned} \text{regret}(\vec{x}) &= (x_1 - 0) \cdot \Pr[A_0] + \sum_{i=2}^n (x_1 - x_i) \cdot \Pr[A_i] \\ &= x_1 \cdot \theta_1 + \left(\sum_{i=2}^{n-1} (x_1 - x_i) \cdot \frac{1}{i} \cdot (1 - \theta_i)^i \right) \\ &\quad - \left(\sum_{i=2}^{n-1} (x_1 - x_i) \cdot \sum_{k=i+1}^{n-1} \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1 - \theta_k)^k \right), \end{aligned}$$

which matches the lemma statement. $\square$

Next, we further relax the expression provided by Lemma 4.2 to a relatively simple form. We define the following.

Definition 4.3. We use $R_q(\vec{x})$ (where $q = 2, 3, \dots$) to present a relaxation of regret:

$$\begin{aligned} R_q(\vec{x}) &:= x_1 \cdot \theta_1 + \left(\sum_{i=2}^n (x_1 - x_i) \cdot \frac{1}{i} \cdot (1 - \theta_i)^i \right) \\ &\quad - \left(\sum_{i=2}^{q-1} (x_1 - x_i) \cdot \sum_{k=i+1}^n \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1 - \theta_k)^k \right) \\ &\geq \text{regret}(\vec{x}), \end{aligned}$$

where the inequality holds by observing that we define $R_q(\vec{x})$ by dropping several negative terms in $\text{regret}(\vec{x})$.

With the definition of $R_q(\vec{x})$, we give the following Lemma 4.4, which suggests that the hardest instance $\vec{x}$ that maximizes the relaxed regret function $R_q(\vec{x})$ has a simple structure.

Lemma 4.4. For fixed $n, x_1, x_2, \dots, x_{q-1}$, if the pricing curve satisfies that $f(x)$ is non-increasing and convex, the function R_q is maximized when $x_q = x_{q+1} = \dots = x_n$.

PROOF. We calculate $\frac{\partial R_q(\vec{x})}{\partial x_k}$ for $j \leq k \leq n$. Recall that

$$R_q(\vec{x}) = x_1 \cdot \theta_1 + \left(\sum_{i=2}^n (x_1 - x_i) \cdot \frac{1}{i} \cdot (1 - \theta_i)^i \right) - \left(\sum_{i=2}^{q-1} (x_1 - x_i) \cdot \sum_{k=i+1}^n \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1 - \theta_k)^k \right).$$

We have

$$\begin{aligned} \frac{\partial R_q(\vec{x})}{\partial x_k} &= \frac{\partial}{\partial x_k} \left[\left((x_1 - x_k) \cdot \frac{1}{k} - \sum_{i=2}^{q-1} (x_1 - x_i) \cdot \frac{1}{k(k-1)} \right) \cdot (1 - \theta_k)^k \right] \\ &= - \left((x_1 - x_k) - \sum_{i=2}^{q-1} (x_1 - x_i) \cdot \frac{1}{k-1} \right) \cdot (1 - \theta_k)^{k-1} \cdot \theta'_k \\ &\quad - \frac{1}{k} \cdot (1 - \theta_k)^k, \end{aligned}$$

where θ'_k denotes $g'(x_k)$. We note that $\frac{\partial R_q(\vec{x})}{\partial x_k}$ does not depend on $x_{k'}$ for $k' \in [q, n] \setminus \{k\}$.

For $k \in [q, n]$ and $f(1) \leq x \leq x_{q-1}$, define

$$u_k(x) := -k \cdot \left((x_1 - x) - \sum_{i=2}^{q-1} (x_1 - x_i) \cdot \frac{1}{k-1} \right) \cdot g'(x) - (1 - g(x)).$$

Recall that we defined $\theta_k = g(x_k)$ and $\theta'_k = g'(x_k)$. Then, we have

$$u_k(x_k) := -k \cdot \left((x_1 - x_k) - \sum_{i=2}^{q-1} (x_1 - x_i) \cdot \frac{1}{k-1} \right) \cdot \theta'_k - (1 - \theta_k)$$

$$\text{and } \frac{\partial R_q(\vec{x})}{\partial x_k} = \frac{1}{k} \cdot u_k(x_k) \cdot (1 - \theta_k)^{k-1}.$$

We next examine the condition of $\frac{\partial R_q(\vec{x})}{\partial x_k} = 0$, which is equivalent to $u_k(x_k) = 0$. Note that have

$$u'_k(x) = (k+1) \cdot g'(x) - k \cdot \left((x_1 - x) + \sum_{i=2}^{q-1} (x_1 - x_i) \cdot \frac{1}{k-1} \right) \cdot g''(x) < 0.$$

The inequality holds because we have $g'(x) < 0$ and $g''(x) \geq 0$, which is true because we assumed $f(t)$ is strictly decreasing and convex, and $g(x)$ is the inverse function of $f(t)$, and

$$(x_1 - x) - \sum_{i=2}^{q-1} (x_1 - x_i) \cdot \frac{1}{k-1} \geq (x_1 - x_{q-1}) - \frac{q-2}{k-1} (x_1 - x_{q-1}) \geq 0.$$

Therefore, function $u_k(x)$ is decreasing within $[f(1), x_{q-1}]$. Additionally, notice that

$$u_k(f(1)) = -k \cdot \left((x_1 - f(1)) - \sum_{i=2}^{q-1} (x_1 - x_i) \cdot \frac{1}{k-1} \right) \cdot g'(f(1)) \geq 0.$$

Therefore, either equation $u_k(x) = 0$ has a unique solution within $[f(1), x_{q-1}]$, or $u_k(x) > 0$ for every $x \in [f(1), x_{q-1}]$. Let z_k be the unique solution of $u_k(x) = 0$ within $[f(1), x_{q-1}]$. If the solution does not exist, we define $z_k = x_{q-1}$. Then, $\frac{\partial R_q(\vec{x})}{\partial x_k}(x)$ is non-negative for $x \in [f(1), z_k]$, and is non-positive for $x \in (z_k, x_{q-1}]$.

We further observe that

$$\begin{aligned} u_{k+1}(z_k) - u_k(z_k) &= -(x_1 - z_k) \cdot g'(z_k) \\ &\quad + \left(\frac{k+1}{k} - \frac{k}{k-1} \right) \cdot \sum_{i=2}^{q-1} (x_1 - x_i) \cdot g'(z_k) \geq 0, \end{aligned}$$

i.e., $u_{k+1}(z_k) \geq u_k(z_k)$. If $u_k(z_k) = 0$, then we have $u_{k+1}(z_k) \geq 0$, and there must be $z_k \leq z_{k+1}$, because function $u_{k+1}(x)$ is decreasing. Otherwise, we have $z_k = x_{q-1}$, and $u_{k+1}(x_{q-1}) > 0$, so $z_{k+1} = x_{q-1}$ should hold. Therefore, in both cases, we have $z_k \leq z_{k+1}$.

Finally, we finish our proof by showing the following claim: given vector $\vec{x} = (x_1, \dots, x_n \geq f(1))$, there exists a vector $\vec{w} = (w_1, \dots, w_n)$ that satisfies the following conditions:

- For $k \in [q-1]$, $w_k = x_k$.
- $w_q = w_{q+1} = \dots = w_n$.
- $R_q(\vec{x}) \leq R_q(\vec{w})$.

We construct $\vec{w}$ by discussing the following three cases. fig. 2 illustrates the relative rankings among different variables.

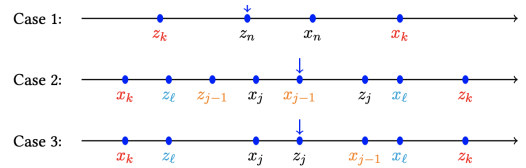


Figure 2: Rankings of different variables in three cases.

Case 1: $z_n \leq x_n$. In this case, we set $w_q = \dots = w_n = z_n$. Since $z_k \leq z_n \leq x_n \leq x_k$ for all $k \in [q, n-1]$, there must be $\frac{\partial R_q(\vec{x})}{\partial x_k}(x) \leq 0$ for $k \in [q, n]$ and $x \in [z_n, x_k]$. Therefore, we have

$$\begin{aligned} R(\vec{w}) - R(\vec{x}) &= \sum_{k=q}^{n-1} \int_{x_k}^{z_{n-1}} \frac{\partial R_q(\vec{x})}{\partial x_k}(x) dx \\ &= - \sum_{k=q}^{n-1} \int_{z_{n-1}}^{x_k} \frac{\partial R_q(\vec{x})}{\partial x_k}(x) dx \geq 0. \end{aligned}$$

The first equality is feasible because the function $\frac{\partial R_q(\vec{x})}{\partial x_k}$ does not depend on $k' \in [q, n] \setminus \{k\}$, i.e., changing the value of $x_{k'}$ does not change the function $\frac{\partial R_q(\vec{x})}{\partial x_k}$.

Case 2: Let $j = \min\{k \in [q, n] : z_k > x_k\}$; j satisfies the condition $x_{j-1} < z_j$. We first note that if we are not in Case 1, at least we have $n \in \{k \in [5, n] : z_k > x_k\}$, and thus j is well defined. In this case, we set $w_q = w_{q+1} = \dots = w_n = x_{j-1}$. For $k \in [j, n]$, we have

$$z_k \geq z_j > x_{j-1} \geq x_j \geq x_k.$$

Therefore, $\frac{\partial R_q(\vec{x})}{\partial x_k}(x) \geq 0$ for $k \in [j, n]$ and $x \in [x_k, x_{j-1}]$. On the other hand, for $\ell \in [q, j-1]$, we have

$$z_\ell \leq z_{j-1} \leq x_{j-1} \leq x_\ell,$$

where the inequality $z_{j-1} \leq x_{j-1}$ is from the assumption that j is the minimum index that satisfies $z_j > x_j$. Therefore, we have

$\frac{\partial R_q(\vec{x})}{\partial x_\ell}(x) \leq 0$ for $\ell \in [q, j-1]$ and $x \in [x_{j-1}, x_\ell]$. Then,

$$R(\vec{w}) - R(\vec{x}) = \sum_{k=j}^n \int_{x_k}^{x_{j-1}} \frac{\partial R_q(\vec{x})}{\partial x_k}(x) dx - \sum_{\ell=q}^{j-1} \int_{x_{j-1}}^{x_\ell} \frac{\partial R_q(\vec{x})}{\partial x_\ell}(x) dx \geq 0.$$

Case 3: Let $j = \min\{k \in [q, n] : z_k > x_k\}$; j satisfies that $z_j \leq x_{j-1}$. In this case, we set $w_q = w_{q+1} = \dots = w_{n-1} = z_j$. For $k \in [j, n]$, we have

$$z_k \geq z_j > x_j \geq x_k.$$

Therefore, $\frac{\partial R_q(\vec{x})}{\partial x_k}(x) \geq 0$ for $k \in [j, n]$ and $x \in [x_k, z_j]$. On the other hand, for $\ell \in [q, j-1]$, we have

$$z_\ell \leq z_j \leq x_{j-1} \leq x_\ell.$$

Therefore, we have $\frac{\partial R_q(\vec{x})}{\partial x_\ell}(x) \leq 0$ for $\ell \in [q, j-1]$ and $x \in [z_j, x_\ell]$. Then,

$$R(\vec{w}) - R(\vec{x}) = \sum_{k=j}^n \int_{x_k}^{z_j} \frac{\partial R_q(\vec{x})}{\partial x_k}(x) dx - \sum_{\ell=q}^{j-1} \int_{z_j}^{x_\ell} \frac{\partial R_q(\vec{x})}{\partial x_\ell}(x) dx \geq 0.$$

In summary, for any vector $\vec{x}$ with fixed $n, x_1, \dots, x_{q-1}$, we can always update the values $x_q, \dots, x_n$ to make sure that $x_q = \dots = x_n$ without decreasing the value of $R(\vec{x})$. This implies that $R(\vec{x})$ is maximized when $x_q = \dots = x_n$ and hence finishes the proof. $\square$

With Lemma 4.4, it is sufficient to consider $R_q(\vec{x})$ under the assumption that $x_q = x_{q+1} = \dots = x_n$. Inspired by this, we have the following definition.

Definition 4.5. We define $\tilde{R}_q(\vec{x}) (q = 2, 3, \dots)$ to present a further relaxation of regret:

$$\begin{aligned} \tilde{R}_q(\vec{x}) := & x_1 \cdot \theta_1 + \left(\sum_{i=2}^{q-1} (x_1 - x_i) \cdot \frac{1}{i} \cdot (1 - \theta_i)^i \right) \\ & + \left(\sum_{i=q}^{\infty} (x_1 - x_q) \cdot \frac{1}{i} \cdot (1 - \theta_q)^i \right) \\ & - \left(\sum_{i=2}^{q-1} (x_1 - x_i) \cdot \sum_{k=i+1}^{q-1} \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1 - \theta_k)^k \right) \\ & - \left(\sum_{i=2}^{q-1} (x_1 - x_i) \cdot \sum_{k=q}^{\infty} \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1 - \theta_q)^k \right), \end{aligned}$$

where the definition comes from taking $x_q = \dots = x_n$ in the function $R_q(\vec{x})$, and we further take $n \rightarrow \infty$, as one can observe that the function (subject to $x_q = \dots = x_n$) is maximized when $n \rightarrow \infty$.

Note that Lemma 4.4 guarantees that the maximum of $\tilde{R}_q(\vec{x})$ is at least the maximum of $\text{regret}(\vec{x})$. Therefore, we reduce the problem of finding the hardest instance for Algorithm 2 from a general n -parameters problem to a q -parameters problem. As we assumed that $n \geq 10$, it would always be feasible to consider $q = 2, 3, \dots, 10$. We will further show in the following subsection that taking a constant q is already sufficient to achieve a good regret upper bound.

4.2 Best-Only Pricing Curve Beats Pricing Curve

Now, we show the power of the relaxation we discussed in Section 4.1. We first show that even when $q = 2$, it already beats the 0.25 lower bound of Algorithm 1 via pricing curve.

Theorem 4.6. *Algorithm 2 with pricing curve $f(t) = \exp(-t/c)$, $c = 0.611$ has regret at most 0.23.*

PROOF. We prove Theorem 4.6 by showing that $\tilde{R}_2(\vec{x})$ for $f(t)$ defined in Theorem 4.6 is upper-bounded by 0.23. By definition,

$$\begin{aligned} \tilde{R}_2(\vec{x}) &= x_1 \cdot \theta_1 + (x_1 - x_2) \cdot \sum_{i=2}^{\infty} \frac{1}{i} \cdot (1 - \theta_2)^i \\ &= x_1 \cdot \theta_1 + (x_1 - x_2) (-\ln \theta_2 - (1 - \theta_2)), \end{aligned}$$

where the last equality uses the Taylor series of $\ln(1-z) = -\sum_{k=1}^{\infty} \frac{z^k}{k}$ for $z \in [0, 1)$.

Take $g(x)$ to be the inverse function of $f(t)$. Then, we have $g(x) = -c \cdot \ln x$. Plugging $\theta_1 = g(x_1)$ into the above equation and taking the partial derivative for x_1 , we have

$$\frac{\partial \tilde{R}_2(\vec{x})}{\partial x_1} = -c \ln x_1 - c - \ln \theta_2 - (1 - \theta_2).$$

Next, we discuss the following two cases:

Case 1: $-c - \ln \theta_2 - (1 - \theta_2) > 0$. In this case, $\frac{\partial \tilde{R}_2(\vec{x})}{\partial x_1}$ is non-negative. Therefore, it's sufficient to only consider $x_1 = 1$ (and therefore $\theta_1 = 0$), and we aim to solve

$$\max_{x_2 \geq f(1)} (1 - x_2) (-\ln \theta_2 - (1 - \theta_2)), \text{ s.t. } -c - \ln \theta_2 - (1 - \theta_2) > 0,$$

where the constraint $x_2 \geq f(1)$ is applied due to Observation 4.1. Since the above optimization problem only involves single parameter, it's easy to numerically check that the problem is maximized when $x_2 \approx 0.7448$, and the corresponding objective is bounded by 0.229.

Case 2: $-c - \ln \theta_2 - (1 - \theta_2) \leq 0$. In this case, function $\tilde{R}_2(\vec{x})$ is maximized when $\frac{\partial \tilde{R}_2(\vec{x})}{\partial x_1} = 0$, which gives

$$x_1 = \exp\left(\frac{-c - \ln \theta_2 - (1 - \theta_2)}{c}\right).$$

Then, the problem becomes a single-parameter optimization problem, which aims at maximizing $\tilde{R}_2(\vec{x})$ with x_1 defining above under the constraints that $-c - \ln \theta_2 - (1 - \theta_2) \leq 0$ and $x_1 \geq f(1)$ (due to Observation 4.1). It's easy to numerically check that the problem is maximized when $x_2 \approx 0.3303$, the corresponding value of x_1 is approximately 0.4132, and the objective is also bounded by 0.229. $\square$

Theorem 4.6 suggests that taking $q = 2$ is already sufficient to provide a simple analysis for Algorithm 2 that gets a regret upper bound strictly better than 0.25. Furthermore, by taking a slightly larger q together with a computer-aided search to find the maximum value of $\tilde{R}_q(\vec{x})$, we are able to give an improved upper bound for the regret of Algorithm 2. Formally, we give the following theorem:

Theorem 4.7. *The best-only pricing curve $f(x) = \exp(-x/c)$ with $c = 0.472$ has regret ≤ 0.190 .*

The proof of Theorem 4.7 takes $q = 5$. We defer the proof of Theorem 4.7 to Section A.1.

Finally, we also provide a lower bound for the class of best-only pricing curves:

Theorem 4.8. *No best-only pricing curve (Algorithm 2) can achieve a regret less than 0.171.*

Due to space limit, we defer the proof of Theorem 4.8 to the full version of the paper.

5 Regret Lower Bound for General Algorithms

In this section, we provide a general regret lower bound of the secretary problem for all algorithms. To be specific, we prove the following:

Theorem 5.1. *For secretary problem with regret minimization as the objective, no algorithm can achieve a regret less than 0.152.*

Our main idea of proving Theorem 5.1 is to apply Yao's minimax principle, which states that the worst-case regret of any (possibly random) algorithm is at least the optimal regret of a deterministic algorithm on a random input. After presenting the hard random input, we show that the optimal algorithm for this hard instance can be explicitly defined and runs in polynomial time. We then show that the optimal algorithm achieves a regret greater than 0.152. The detailed proof of Theorem 5.1 is deferred to Section B.

6 Revenue Maximization with Regret Minimization Objective

So far, we have discussed regret minimization for the secretary problem. In this section, we further discuss the *revenue* maximization variant of the online stopping problem with minimizing regret as the objective. In contrast with welfare maximization in earlier sections, here, a seller is selling one item to n buyers that arrive sequentially. Buyer i has value $x_i \in [0, 1]$. At the time that buyer i arrives, the seller needs to post a price p_i . If $x_i \geq p_i$, buyer i buys the item, and the seller gets p_i as revenue, ending the process. As before, the goal is to minimize the *regret*, defined as the difference between (sometimes the expectation of) the best possible revenue $\max_{i \in [n]} x_i$, and the expected revenue collected by the seller.

There are different modeling choices about values and arrivals. Values can be picked by an adversary, or can be drawn stochastically from same/different known distributions. Similarly, the arrival order can be uniformly random or chosen by an adversary. For the positive result of the problem, we show that there exists an algorithm that achieves $1/e$ regret even in the strongest adversary setting.

Theorem 6.1. *For revenue maximization variant of online stopping problem, there exists an algorithm that achieves $1/e$ regret when buyers have adversarial values.*

To prove Theorem 6.1, we present an algorithm – randomized uniform price (algorithm 3) – that can achieve optimal regret of $1/e$ in all of these revenue maximization settings. This algorithm was first discussed in [44] for regret minimization for *welfare* with adversarial arrival order. We further observe that the same algorithm

also gives the same regret guarantee for the revenue maximization setting.

Algorithm 3: Randomized Uniform Price

```

Let  $\alpha \sim U[-1, 0]$ , and set uniform price  $p \leftarrow e^\alpha$ 
for  $\tau = 0 \rightarrow 1$  where  $x_i$  arrives at  $t_i = \tau$  do
    if  $x_i \geq p$  then
        | Pick  $x_i$  and exit
    else
        | Skip  $x_i$ 
    end
end

```

Now, we show that Algorithm 3 is the desired algorithm for Theorem 6.1.

PROOF OF THEOREM 6.1. Let $z := \max\{x_1, \dots, x_n\}$ be the optimal revenue in hindsight. There are two cases:

- Case (1): $z \leq 1/e$, and the regret is at most $z \leq 1/e$.
- Case (2): $z > 1/e$. In this case, the item is sold if and only if $z \geq p$, which is equivalent to $\alpha \leq \ln z$. The seller gets revenue of $p = e^\alpha$ if the item is sold. Therefore, the expected revenue of algorithm 3 is

$$\int_{-1}^{\ln z} e^\alpha d\alpha = z - \frac{1}{e}.$$

Therefore, the regret is $z - (z - 1/e) = 1/e$. $\square$

For the negative result of the problem, we provide a matching $1/e$ regret lower bound. We also show the hard instance only contains a single stochastic buyer. Therefore the extra stochastic or random order assumption does not make the problem easier.

Theorem 6.2. *For revenue maximization variant of online stopping problem, there exists an instance in which no algorithm can achieve a regret strictly better than $1/e$ when buyers have adversarial values.*

PROOF. Consider a buyer with valuation $x \sim \mathcal{D}$, where the cumulative distribution function of the distribution $\mathcal{D}$ is

$$F_{\mathcal{D}}(x) = \begin{cases} 0 & x \in \left[0, \frac{1}{e}\right) \\ 1 - \frac{1}{ex} & x \in \left[\frac{1}{e}, 1\right) \\ 1 & x = 1 \end{cases}$$

Observe that an optimal strategy for the seller is to use any price in $[1/e, 1)$ and get expected revenue of

$$p \cdot (1 - F_{\mathcal{D}}(p)) = \frac{1}{e}.$$

On the other hand,

$$\mathbb{E}[x] = \int_0^1 (1 - F(x)) dx = \frac{2}{e}.$$

By linearity of expectation, the optimal regret for the seller is $2/e - 1/e = 1/e$. $\square$

References

- [1] Saeed Alaei. 2014. Bayesian Combinatorial Auctions: Expanding Single Buyer Mechanisms to Many Buyers. *SIAM J. Comput.* 43, 2 (2014), 930–972. doi:10.1137/120878422
- [2] Alessandro Arlotto and Itai Gurvich. 2019. Uniformly Bounded Regret in the Multisecretary Problem. *Stochastic Systems* 9, 3 (2019), 231–260. doi:10.1287/stsy.2018.0028
- [3] Yossi Azar, Ashish Chiplunkar, and Haim Kaplan. 2018. Prophet Secretary: Surpassing the $1-1/e$ Barrier. In *Proceedings of the 2018 ACM Conference on Economics and Computation (EC)*. ACM, 303–318. doi:10.1145/3219166.3219182
- [4] Moshe Babaioff, Nicole Immorlica, David Kempe, and Robert D. Kleinberg. 2018. Matroid Secretary Problems. *J. ACM* 65, 6 (2018), 35:1–35:26. doi:10.1145/3212512
- [5] Moshe Babaioff, Nicole Immorlica, and Robert D. Kleinberg. 2007. Matroids, secretary problems, and online mechanisms. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 434–443. <http://dl.acm.org/citation.cfm?id=1283383.1283429>
- [6] MohammadHossein Bateni, Mohammad Taghi Hajiaghayi, and Morteza Zadimoghaddam. 2013. Submodular secretary problem and extensions. *ACM Trans. Algorithms* 9, 4 (2013), 32:1–32:23. doi:10.1145/2500121
- [7] Omar Besbes, Yash Kanoria, and Akshit Kumar. 2022. The Multi-secretary Problem with Many Types. In *Proceedings of the 23rd ACM Conference on Economics and Computation (EC'22)*. doi:10.1145/3490486.3538371 arXiv:2205.09078.
- [8] Domagoj Bradac, Anupam Gupta, Sahil Singla, and Goran Zuzic. 2020. Robust Algorithms for the Secretary Problem. In *11th Innovations in Theoretical Computer Science Conference (ITCS)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 32:1–32:26. doi:10.4230/LIPICS.ITCS.2020.32
- [9] Robert L. Bray. 2024. Logarithmic Regret in Multisecretary and Online Linear Programs with Continuous Valuations. *Operations Research* 73, 4 (2024), 2188–2203. doi:10.1287/opre.2022.0036
- [10] Archit Bubna and Ashish Chiplunkar. 2023. Prophet Inequality: Order selection beats random order. In *Proceedings of the 24th ACM Conference on Economics and Computation (EC)*. ACM, 302–336. doi:10.1145/3580507.3597687
- [11] Pornpawee Bumpensanti and He Wang. 2020. A Re-solving Heuristic with Uniformly Bounded Loss for Network Revenue Management. *Management Science* 66, 7 (2020), 2993–3009. doi:10.1287/mnsc.2019.3365
- [12] Sourav Chakraborty and Oded Lachish. 2012. Improved competitive ratio for the matroid secretary problem. In *Proceedings of the 23rd Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. Yuval Rabani (Ed.), SIAM, 1702–1712. doi:10.1137/1.9781611973099.135
- [13] Shuchi Chawla, Jason D. Hartline, David L. Malec, and Balasubramanian Sivan. 2010. Multi-parameter mechanism design and sequential posted pricing. In *Proceedings of the 42nd ACM Symposium on Theory of Computing (STOC)*. ACM, 311–320. doi:10.1145/1806689.1806733
- [14] Shuchi Chawla, David L. Malec, and Balasubramanian Sivan. 2015. The power of randomness in Bayesian optimal mechanism design. *Games Econ. Behav.* 91 (2015), 297–317. doi:10.1016/J.GEB.2012.08.010
- [15] Y. S. Chow, Sigaiti Moriguti, Herbert Robbins, and S. Mo Samuels. 1964. Optimal selection based on relative rank (the “secretary problem”). *Israel Journal of mathematics* 2, 2 (1964), 81–90.
- [16] José R. Correa, Andrés Cristi, Laurent Feuilloley, Tim Oosterwijk, and Alexandros Tsigonias-Dimitriadis. 2021. The Secretary Problem with Independent Sampling. In *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 2047–2058. doi:10.1137/1.9781611976465.122
- [17] José R. Correa, Raimundo Saona, and Bruno Ziliotto. 2021. Prophet secretary through blind strategies. *Math. Program.* 190, 1 (2021), 483–521. doi:10.1007/S10107-020-01544-8
- [18] Nedyalko B. Dimitrov and C. Greg Plaxton. 2012. Competitive Weighted Matching in Transversal Matroids. *Algorithmica* 62, 1-2 (2012), 333–348. doi:10.1007/S00453-010-9457-2
- [19] Evgenii Borisovich Dynkin. 1963. The optimum choice of the instant for stopping a Markov process. *Soviet Mathematics* 4 (1963), 627–629.
- [20] Soheil Ehsani, MohammadTaghi Hajiaghayi, Thomas Kesselheim, and Sahil Singla. 2018. Prophet Secretary for Combinatorial Auctions and Matroids. In *Proceedings of the 29th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 700–714. doi:10.1137/1.9781611975031.46
- [21] Hossein Esfandiari, MohammadTaghi Hajiaghayi, Vahid Liaghat, and Morteza Monemizadeh. 2017. Prophet Secretary. *SIAM J. Discret. Math.* 31, 3 (2017), 1685–1701. doi:10.1137/15M1029394
- [22] Tomer Ezra, Michal Feldman, Nick Gravin, and Zhihao Gavin Tang. 2020. Online Stochastic Max-Weight Matching: Prophet Inequality for Vertex and Edge Arrival Models. In *Proceedings of the 21st ACM Conference on Economics and Computation (EC)*. ACM, 769–787. doi:10.1145/3391403.3399513
- [23] Tomer Ezra, Michal Feldman, Nick Gravin, and Zhihao Gavin Tang. 2022. General Graphs are Easier than Bipartite Graphs: Tight Bounds for Secretary Matching. In *Proceedings of the 23rd ACM Conference on Economics and Computation (EC)*. ACM, 1148–1177. doi:10.1145/3490486.3538290
- [24] Michal Feldman, Nick Gravin, and Brendan Lucier. 2015. Combinatorial Auctions via Posted Prices. In *Proceedings of the 26th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 123–135. doi:10.1137/1.9781611973730.10
- [25] Moran Feldman, Ola Svensson, and Rico Zenklusen. 2018. A Simple $O(\log \log \text{rank})$ -Competitive Algorithm for the Matroid Secretary Problem. *Math. Oper. Res.* 43, 2 (2018), 638–650. doi:10.1287/MOOR.2017.0876
- [26] Thomas S. Ferguson. 1989. Who solved the secretary problem? *Statistical science* 4, 3 (1989), 282–289.
- [27] Martin Gardner. 1960. Mathematical Games.
- [28] Giordano Giambartolomei, Frederik Mallmann-Trenn, and Raimundo Saona. 2023. Prophet Inequalities: Separating Random Order from Order Selection. *Math. Oper. Res.* 48, 2 (2023), 1235–1250. arXiv preprint arXiv:2304.04024 (2023).
- [29] Elfarouk Harb. 2023. Fishing For Better Constants: The Prophet Secretary Via Poissonization. *arXiv preprint arXiv:2307.00971* (2023).
- [30] Jason D. Hartline. 2013. Mechanism design and approximation. *Book draft*. October 122, 1 (2013).
- [31] Theodore P. Hill. 1983. Prophet inequalities and order selection in optimal stopping problems. *Proc. Amer. Math. Soc.* 88, 1 (1983), 131–137.
- [32] Theodore P. Hill and Robert P. Kertz. 1981. Additive comparisons of stop rule and supremum expectations of uniformly bounded independent random variables. *Proc. Amer. Math. Soc.* 83, 3 (1981), 582–585.
- [33] Theodore P. Hill and Robert P. Kertz. 1982. Comparisons of stop rule and supremum expectations of iid random variables. *The Annals of Probability* (1982), 336–345.
- [34] Theodore P. Hill and Robert P. Kertz. 1983. Stop rule inequalities for uniformly bounded sequences of random variables. *Trans. Amer. Math. Soc.* 278, 1 (1983), 197–207.
- [35] Sungjin Im and Yajun Wang. 2011. Secretary Problems: Laminar Matroid and Interval Scheduling. In *Proceedings of the 22nd Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 1265–1274. doi:10.1137/1.9781611973082.96
- [36] Haim Kaplan, David Naor, and Danny Raz. 2020. Competitive Analysis with a Sample and the Secretary Problem. In *Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 2082–2095. doi:10.1137/1.9781611975994.128
- [37] Thomas Kesselheim, Klaus Radke, Andreas Tönnis, and Berthold Vöcking. 2013. An Optimal Online Algorithm for Weighted Bipartite Matching and Extensions to Combinatorial Auctions. In *21st Annual European Symposium (ESA)*, Vol. 8125. Springer, 589–600. doi:10.1007/978-3-642-40450-4_50
- [38] Robert D. Kleinberg and S. Matthew Weinberg. 2019. Matroid prophet inequalities and applications to multi-dimensional mechanism design. *Games Econ. Behav.* 113 (2019), 97–115. doi:10.1016/J.GEB.2014.11.002
- [39] Nitish Korula and Martin Pál. 2009. Algorithms for Secretary Problems on Graphs and Hypergraphs. In *Proceedings of the 36th International Colloquium on Automata, Languages and Programming (ICALP)*, Vol. 5556. Springer, 508–520. doi:10.1007/978-3-642-02930-1_42
- [40] Ulrich Krengel and Louis Sucheston. 1977. Semiamarts and finite values. *Bull. Am. Math. Soc.* (1977).
- [41] Ulrich Krengel and Louis Sucheston. 1978. On semiamarts, amarts, and processes with finite value. *Advances in Prob* 4 (1978), 197–266.
- [42] Oded Lachish. 2014. $O(\log \log \text{rank})$ Competitive Ratio for the Matroid Secretary Problem. In *Proceedings of the 55th IEEE Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE Computer Society, 326–335. doi:10.1109/FOCS.2014.42
- [43] Denis V. Lindley. 1961. Dynamic programming and decision theory. *Journal of the Royal Statistical Society: Series C (Applied Statistics)* 10, 1 (1961), 39–51.
- [44] Virtual Institute on Prophet Inequalities. 2021.
- [45] Aviad Rubinstein. 2016. Beyond matroids: secretary problem and prophet inequality with general constraints. In *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing (STOC)*. ACM, 324–332. doi:10.1145/2897518.2897540
- [46] Aviad Rubinstein and Sahil Singla. 2017. Combinatorial Prophet Inequalities. In *Proceedings of the 28th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 1671–1687. doi:10.1137/1.9781611974782.110
- [47] Ester Samuel-Cahn. 1984. Comparison of threshold stop rules and maximum for independent nonnegative random variables. *the Annals of Probability* (1984), 1213–1216.
- [48] Alberto Vera, Siddhartha Banerjee, and Itai Gurvich. 2021. Online Allocation and Pricing: Constant Regret via Bellman Inequalities. *Operations Research* 69, 3 (2021), 821–840. doi:10.1287/opre.2020.2061
- [49] Qiqi Yan. 2011. Mechanism Design via Correlation Gap. In *Proceedings of the 22nd Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 710–719. doi:10.1137/1.9781611973082.56

A Omitted Proofs in Section 4

A.1 Proof of Theorem 4.7

Theorem 4.7. *The best-only pricing curve $f(x) = \exp(-x/c)$ with $c = 0.472$ has regret ≤ 0.190 .*

PROOF. Recall that

$$\begin{aligned} \tilde{R}_5(\vec{x}) := & x_1 \cdot \theta_1 + \left(\sum_{i=2}^4 (x_1 - x_i) \cdot \frac{1}{i} \cdot (1 - \theta_i)^i \right) \\ & + \left(\sum_{i=5}^{\infty} (x_1 - x_5) \cdot \frac{1}{i} \cdot (1 - \theta_5)^i \right) \\ & - \left(\sum_{i=2}^4 (x_1 - x_i) \cdot \sum_{k=i+1}^4 \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1 - \theta_k)^k \right) \\ & - \left(\sum_{i=2}^4 (x_1 - x_i) \cdot \sum_{k=5}^{\infty} \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1 - \theta_5)^k \right). \end{aligned}$$

By applying Taylor series of

- $\ln(1 - z) = -\sum_{k=1}^{\infty} \frac{z^k}{k}$, for $z \in [0, 1)$, and
- $(1 - z) \ln(1 - z) = -z + \sum_{k=2}^{\infty} \frac{z^k}{k(k-1)}$, for $z \in [0, 1)$,

for $z = 1 - \theta_5$, we have

$$\begin{aligned} \tilde{R}_5(\vec{x}) = & x_1 \cdot \theta_1 + \left(\sum_{i=2}^4 (x_1 - x_i) \cdot \frac{1}{i(i-1)} \cdot (1 - \theta_i)^i \right) \\ & - (x_1 - x_5) \left(\ln(\theta_5) + (1 - \theta_5) + \frac{(1 - \theta_5)^2}{2} + \frac{(1 - \theta_5)^3}{3} + \frac{(1 - \theta_5)^4}{4} \right) \\ & - \left(\sum_{i=2}^4 (x_1 - x_i) \right) \\ & \cdot \left(\theta_5 \ln(\theta_5) + (1 - \theta_5) - \frac{(1 - \theta_5)^2}{2} - \frac{(1 - \theta_5)^3}{6} - \frac{(1 - \theta_5)^4}{12} \right). \end{aligned}$$

It can be shown via computer-aided search that the maximum of above function is at most 0.190, which proves Theorem 4.7. $\square$

Finding the maximum of $\tilde{R}_5(\vec{x})$ via heuristic grid search. We briefly discuss the search algorithm we use for finding the maximum of $\tilde{R}_5(\vec{x})$. The goal of Theorem 4.7 is to find a near-optimal function f such that the value of $\max_{\vec{x}} \tilde{R}_5(\vec{x})$ is approximately maximized. To resolve this optimization problem, we use a two-layer heuristic search. The outer layer searches over a certain class of parameterized functions (e.g., function class $f(t) = \exp(-t/c)$, function class $f(t) = (1 - x)^c$, and 10-folded piecewise linear functions). Once the curve is fixed, we use a second heuristic search in the inner layer to find the optimal parameters in $\vec{x}$ that maximize $\tilde{R}_5(\vec{x})$.

After completing the above experiment, we find that $f(t) = \exp(-t/c)$ with $c = 0.472$ is the near-optimal curve among all the classes of curves we have tried, with an approximate regret upper bound of 0.188, given by the inner-layer heuristic search algorithm. However, this regret cannot be directly used as a feasible regret upper bound. To verify the true regret upper bound of the curve $f(t) = \exp(-t/0.472)$, we calculate its Lipschitz coefficient, which leads to the following Lemma A.1:

Lemma A.1. *Let $\vec{x}$ and $\vec{y} = (y_1, y_2, y_3, y_4, y_5)$ be two vectors. If we have $x_1 \geq y_1, x_2 \leq y_2, x_3 \leq y_3, x_4 \leq y_4, x_5 \leq y_5$, then*

$$\begin{aligned} \tilde{R}_5(\vec{y}) - \tilde{R}_5(\vec{x}) \leq & 0.472(x_1 - y_1) + 0.444(y_2 - x_2) \\ & + 0.195(y_3 - x_3) + 0.128(y_4 - x_4) + 0.294(y_5 - x_5). \end{aligned}$$

We defer the proof of Lemma A.1 to Section A.2, and first show how we apply Lemma A.1 to determine the 0.190 regret upper bound. Our approach is based on a recursive search algorithm. We first initialize the search range $[l_i, r_i] = [f(1), 1]$ for $i \in [5]$, indicating that the search range is a 5-dimension hypercube that contains all vectors satisfying $l_i \leq x_i \leq r_i$ for $i \in [5]$ (recall that the values smaller than $f(1)$ can be omitted, according to Observation 4.1). When the recursive algorithm takes a 5-dimensional hypercube as the input, it first check the value of $\tilde{R}_5(r_1, l_2, l_3, l_4, l_5)$. Then, Lemma A.1 gives an upper bound on all the remaining vectors inside the current search range. If the upper bound is smaller than the target bound 0.190, the recursive algorithm terminates, as it suggests that the function value of all possible vectors in the hypercube is bounded by 0.190; otherwise, the algorithm divides each $[l_i, r_i]$ into $[l_i, (l_i + r_i)/2]$ and $[(l_i + r_i)/2, r_i]$, and all possible combinations form 32 small hypercubes. Then, the algorithm searches each of the 32 small hypercubes recursively.

Note that the search algorithm must terminate if the maximum of $\tilde{R}_5(\vec{x})$ is smaller than 0.190. As the search algorithm we run terminates, it implies the maximum of $\tilde{R}_5(\vec{x})$ is smaller than 0.190.

A.2 proof of Lemma A.1

For simplicity of the analysis, we use the original version of

$$\begin{aligned} \tilde{R}_5(\vec{x}) = & x_1 \cdot \theta_1 + \left(\sum_{i=2}^4 (x_1 - x_i) \cdot \frac{1}{i} \cdot (1 - \theta_i)^i \right) \\ & + \left(\sum_{i=5}^{\infty} (x_1 - x_5) \cdot \frac{1}{i} \cdot (1 - \theta_5)^i \right) \\ & - \left(\sum_{i=2}^4 (x_1 - x_i) \cdot \sum_{k=i+1}^4 \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1 - \theta_k)^k \right) \\ & - \left(\sum_{i=2}^4 (x_1 - x_i) \cdot \sum_{k=5}^{\infty} \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1 - \theta_5)^k \right). \end{aligned}$$

to prove Lemma A.1. Note that to prove Lemma A.1, it's sufficient to prove that $\frac{\partial \tilde{R}_5(\vec{x})}{\partial x_1} \geq -0.472, \frac{\partial \tilde{R}_5(\vec{x})}{\partial x_i} \leq 0.444, 0.195, 0.128, 0.294$ for $i = 2, 3, 4, 5$, respectively. Then, a multiple integral is sufficient to prove Lemma A.1.

Bounding $\frac{\partial \tilde{R}_5(\vec{x})}{\partial x_1}$. We have

$$\begin{aligned} \frac{\partial \tilde{R}_5(\vec{x})}{\partial x_1} = & -c + \theta_1 + \sum_{i=2}^4 \frac{1}{i} \cdot (1 - \theta_i)^i + \sum_{i=5}^{\infty} \frac{1}{i} \cdot (1 - \theta_5)^i \\ & - \sum_{k=2}^4 (k-2) \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1 - \theta_k)^k \\ & - 3 \sum_{k=5}^{\infty} \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1 - \theta_5)^k \end{aligned}$$

$$\begin{aligned}
&= -c + \theta_1 + \sum_{i=2}^4 \frac{1}{i(i-1)} \cdot (1-\theta_i)^i + \sum_{i=5}^{\infty} \frac{i-4}{i(i-1)} \cdot (1-\theta_5)^i \\
&\geq -c = -0.472.
\end{aligned}$$

Bounding $\frac{\partial \tilde{R}_5(\vec{x})}{\partial x_2}$. We have

$$\begin{aligned}
\frac{\partial \tilde{R}_5(\vec{x})}{\partial x_2} &= -\frac{1}{2} \cdot (1-\theta_2)^2 - (x_1-x_2)(1-\theta_2)\theta'_2 \\
&\quad + \sum_{k=3}^4 \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1-\theta_k)^k + \sum_{k=5}^{\infty} \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1-\theta_5)^k \\
&\leq -\frac{1}{2} \cdot (1-\theta_2)^2 + (1-x_2)(1-\theta_2) \frac{c}{x_2} \\
&\quad + \frac{1}{6} \cdot (1-\theta_2)^3 + \frac{1}{12} \cdot (1-\theta_2)^4 + \frac{1}{4} \cdot (1-\theta_2)^5 \leq 0.444,
\end{aligned}$$

where the first inequality is from the observation that

$$\begin{aligned}
\sum_{k=5}^{\infty} \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1-\theta_5)^k &\leq \sum_{k=5}^{\infty} \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1-\theta_5)^5 \\
&= \frac{1}{4} (1-\theta_5)^5, \tag{1}
\end{aligned}$$

and the last inequality can be determined via solving a single-parameter optimization problem.

Bounding $\frac{\partial \tilde{R}_5(\vec{x})}{\partial x_3}$. We have

$$\begin{aligned}
\frac{\partial \tilde{R}_5(\vec{x})}{\partial x_3} &= -\frac{1}{3} \cdot (1-\theta_3)^3 - (x_1-x_3)(1-\theta_3)^2\theta'_3 + \frac{1}{12} \cdot (1-\theta_4)^4 \\
&\quad + \left((x_1-x_2) \cdot \frac{1}{2} \cdot (1-\theta_3)^2 \cdot \theta'_3 \right) \\
&\quad + \sum_{k=5}^{\infty} \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1-\theta_5)^k \\
&\leq -\frac{1}{3} \cdot (1-\theta_3)^3 + (1-x_3)(1-\theta_3)^2 \frac{c}{x_3} \\
&\quad + \frac{1}{12} \cdot (1-\theta_3)^4 + \frac{1}{4} \cdot (1-\theta_3)^5 \\
&\leq 0.195,
\end{aligned}$$

where the first inequality follows from Equation (1) together with the fact that $(x_1-x_2) \cdot \frac{1}{2} \cdot (1-\theta_3)^2 \cdot \theta'_3 \leq 0$, and the last inequality can be given via solving a single-parameter optimization problem.

Bounding $\frac{\partial \tilde{R}_5(\vec{x})}{\partial x_4}$. We have

$$\begin{aligned}
\frac{\partial \tilde{R}_5(\vec{x})}{\partial x_4} &= -\frac{1}{4} \cdot (1-\theta_4)^4 - (x_1-x_4)(1-\theta_4)^3\theta'_4 \\
&\quad + \left((2x_1-x_2-x_3) \cdot \frac{1}{3} \cdot (1-\theta_4)^3\theta'_4 \right) \\
&\quad + \sum_{k=5}^{\infty} \left(\frac{1}{k-1} - \frac{1}{k} \right) \cdot (1-\theta_5)^k \\
&\leq -\frac{1}{4} \cdot (1-\theta_4)^4 + (1-x_4)(1-\theta_4)^3 \frac{c}{x_4} + \frac{1}{4} (1-\theta_4)^5 \\
&\leq 0.128,
\end{aligned}$$

where the first inequality follows from Equation (1) together with the fact that $(2x_1-x_2-x_3) \cdot \frac{1}{3} \cdot (1-\theta_4)^3\theta'_4 \leq 0$, and the last inequality can be given via solving a single-parameter optimization problem.

Bounding $\frac{\partial \tilde{R}_5(\vec{x})}{\partial x_5}$. We have

$$\begin{aligned}
\frac{\partial \tilde{R}_5(\vec{x})}{\partial x_5} &= -\sum_{i=5}^{\infty} \frac{1}{i} \cdot (1-\theta_5)^i - \sum_{i=5}^{\infty} (x_1-x_5) \cdot (1-\theta_5)^{i-1} \cdot \theta'_5 \\
&\quad + \left((3x_1-x_2-x_3-x_4) \cdot \sum_{k=5}^{\infty} \frac{1}{k-1} \cdot (1-\theta_5)^{k-1} \cdot \theta'_5 \right) \\
&\leq -\sum_{i=5}^{\infty} \frac{1}{i} \cdot (1-\theta_5)^i + \sum_{i=5}^{\infty} (1-x_5) \cdot (1-\theta_5)^{i-1} \cdot \frac{c}{x_5} \\
&= \left(\ln(\theta_5) + (1-\theta_5) + \frac{(1-\theta_5)^2}{2} + \frac{(1-\theta_5)^3}{3} - \frac{(1-\theta_5)^4}{4} \right) \\
&\quad + (1-x_5) \cdot (1-\theta_5)^4 \cdot \frac{c}{x_5} \cdot \frac{1}{\theta_5} \\
&\leq 0.294,
\end{aligned}$$

where the first inequality follows from Equation (1) together with the fact that $(3x_1-x_2-x_3-x_4) \cdot \sum_{k=5}^{\infty} \frac{1}{k-1} \cdot (1-\theta_5)^{k-1} \cdot \theta'_5 \leq 0$, and the last inequality can be given via solving a single-parameter optimization problem.

B Omitted Proofs in Section 5

In this section, we prove Theorem 5.1, which gives a regret lower bound for all possible algorithms, via applying Yao's minimax principle. Theorem 5.1 is restated below for convenience.

Theorem 5.1. *For secretary problem with regret minimization as the objective, no algorithm can achieve a regret less than 0.152.*

Next, we give the hard instance for proving Theorem 5.1, which is defined in Example B.1.

Example B.1. Let $V = \{a, b, c, 0\}$, where $a = 1$, $b = 0.59$, and $c = 0.38$. The random example B.1 is constructed in the following way, with $M := 10^5$ being a sufficiently large integer.

- With probability $p_1 = 0.46$, the input is I_1 , which contains $V_1 = \{a, b, c\}$ and $M-3$ values of 0.
- With probability $p_2 = 0.27$, the input is I_2 , which contains $V_2 = \{b, c\}$ and $M-2$ values of 0.
- With probability $p_3 = 0.27$, the input is I_3 , which contains $V_3 = \{c\}$ and $M-1$ values of 0.

Now, we prove Theorem 5.1 by showing that the optimal algorithm for Example B.1 has the form of Algorithm 4, and further showing that the regret of Algorithm 4 is at least 0.152.

PROOF OF THEOREM 5.1. The main idea of our proof is to show that the optimal decision function $A^*(S, v, i)$ can be calculated via a backward dynamic programming. Then, the optimality of $A^*(S, v, i)$ guarantees Algorithm 4 is the optimal algorithm for Example B.1.

We first introduce the following two notations:

- $E(k, S, i)$: we use $E(k, S, i)$ to represent the expected outcome if we run Algorithm 4 starting from the i -th arrival, assuming the input of the instance is I_k , subset $S \subseteq V_k$ has arrived before the i -th arrival, and the remaining $V_k \setminus S$ will arrive at the arrival slot $i, i+1, \dots, M$ uniformly at random. We note

Algorithm 4: Optimal Algorithm for $\mathcal{I}$

Let $A^*(S, v, i): 2^V \times V \times [M] \rightarrow \{0, 1\}$ be the optimal decision function

Initialize $S \leftarrow \emptyset$

for $i = 1 \rightarrow M - 1$ **do**

 Observe the i -th arrival y_i

if $y_i = 0$ **then continue;**

if $A^*(S, y_i, i) = 1$ **then**

 Pick y_i and **exit**

else

 Skip y_i and update $S \leftarrow S \cup \{y_i\}$

end

end

Pick y_M if the algorithm has not picked any value

that $E(k, S, i)$ is not well defined when S is not a subset of V_k , or $|V_k \setminus S| > M - i + 1$. For these cases, we set the value of $E(k, S, i)$ be $-\infty$ to mark that this status is invalid.

We will show with induction that the value of $E(k, S, i)$ can be computed explicitly.

- $P_k^{S,v,i}$: we define

$$P_k^{S,v,i} := \begin{cases} p_k \cdot \frac{P(i-1, |S|) \cdot P(M-i, |V_k| - |S| - 1)}{P(M, V_k)} & S \cup \{v\} \subseteq V_k \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

to be the probability that the following events happen simultaneously: the input for Algorithm 4 is $\mathcal{I}_k$; subset $S \subseteq V_k$ has arrived before the i -th arrival; value $v \in V_k$ is the i -th arrival.

Now we use backward induction to show algorithm 4 is the optimal algorithm for example B.1. Simultaneously, we show the optimal $A^*(S, v, i)$ is defined by a backward dynamic programming process.

The base case is $i = M$. In this case, the optimal strategy is to take any arriving value, i.e., setting $A^*(S, v, M) = 1$ for every S and v is sufficient. Besides, we have $E(k, S, M) = v$ when $|V_k \setminus S| = 1$, where v is the only value in $V_k \setminus S$, and $E(k, S, M) = 0$ when $S = V_k$.

For the induction step, we consider the optimal strategy at the i -th arrival, assuming the optimal algorithm for example B.1 follows Algorithm 4 for arrival slots $i+1, i+2, \dots, M$. We also assume value $E(k, S, j)$ is computable for $j = i+1, i+2, \dots, M$.

We first decide $A^*(S, 0, i)$. Note that there is no incentive to pick value 0 when $i \neq M$. Therefore, we set $A^*(S, 0, i) = 0$ for all S .

Now consider the case that a value $v \neq 0$ arrives at slot i . There are two possible options. If we take v , the algorithm gets value v . On the other hand, if we skip v , the fact that subset $S \cup \{v\}$ has arrived gives a conditional distribution for the future arrivals, and therefore the expected outcome of algorithm 4 in the future is computable. Specifically, if we skip v at slot i , the expected outcome of algorithm 4 is exactly

$$\sum_{k=1}^3 \frac{P_k^{S,v,i}}{\sum_{j \in [3]} P_j^{S,v,i}} \cdot E(k, S \cup \{v\}, i). \quad (3)$$

With the help of eq. (3), the optimal strategy for the i -th arrival is clear: if a value $v \in \{a, b, c\}$ becomes the i -th arrival, we take it and set $A^*(S, v, i) = 1$ when v is greater than eq. (3); otherwise, the optimal strategy is to skip v , and we set $A^*(S, v, i) = 0$ in this case.

Finally, we show the value of $E(k, S, i)$ is computable to finish the induction step. The value of $E(k, S, i)$ is explicitly given by the following expression:

$$E(k, S, i) = \sum_{v \in V_k \setminus S} \frac{1}{M-i+1} \cdot \left(1[A^*(S, v, i) = 1] \cdot v + 1[A^*(S, v, i) = 0] \cdot E(k, S \cup \{v\}, i+1) \right).$$

The backward induction shows that we can determine the optimal algorithm for example B.1 by calculating the value of $A^*(S, v, i)$ and $E(k, S, i)$ step-by-step via a backward dynamic programming from $i = M$ to $i = 1$. Therefore, algorithm 4 is the optimal algorithm for example B.1. Following the calculation procedure described above, we find the optimal decision function $A^*(S, v, i)$ as follows:

- $A^*(S, a, i) = 1$ for all $S \ni a, i \in [M]$.
- $A^*(S, v, i) = 0$ for all $S \ni a, v \in \{b, c\}, i \in [M]$.
- $A^*(\emptyset, b, i) = 1$ for $i \in [29396, M]$, while $A^*(\emptyset, b, i) = 0$ for $i \in [1, 29395]$.
- $A^*(\emptyset, c, i) = 1$ for $i \in [40051, M]$, while $A^*(\emptyset, c, i) = 0$ for $i \in [1, 40050]$.
- $A^*(\{b\}, c, i) = 1$ for $i \in [64026, M]$, while $A^*(\{b\}, c, i) = 0$ for $i \in [1, 64025]$.
- $A^*(\{c\}, b, i) = 1$ for $i \in [15538, M]$, while $A^*(\{c\}, b, i) = 0$ for $i \in [1, 15537]$.

It can be further computed that the regret for example B.1 with this optimal function $A^*(S, v, i)$ has regret at least 0.1529, which finishes the proof of Theorem 5.1. $\square$